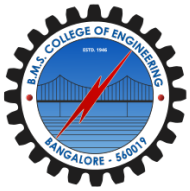
B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bangalore - 560019



Bio Inspired Algorithms AAT on

Course Code: **23DS7BSBIO**

Plant Disease Detection using Transfer-Learning CNN Optimized with Differential Evolution

in fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING

in

Computer Science & Engineering (Data Science)

by

Aryanraj S Singh	**1BM23CD008**
Richik Chaudhuri	**1BM23CD050**
Suraj B	**1BM23CD061**
Sushant Deo	**1BM23CD062**

Under the Guidance of

Manoj Kumar S

Assistant Professor

Department of CSE (Data Science), B.M.S. College of Engineering

AY 2026–2027

Team 10

B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Department of Computer Science & Engineering (Data Science)

CERTIFICATE

This is to certify that the project entitled "Plant Disease Detection using Transfer-Learning CNN Optimized with Differential Evolution" is a bona fide work carried out by the following students for the course Bio Inspired Algorithms (Course Code: **23DS7BSBIO**). It is certified that all corrections and suggestions indicated during the internal assessments have been incorporated in the final report submitted to the departmental library. The project report has been approved as it meets the academic requirements prescribed for the Bachelor of Engineering Degree in respect of project work.

Sl. No.	Student Name	USN	Student's Signature
1	Aryanraj S Singh	1BM23CD008	
2	Richik Chaudhuri	1BM23CD050	
3	Suraj B	1BM23CD061	
4	Sushant Deo	1BM23CD062	

Signature of the Guide

Manoj Kumar S
Assistant Professor

Signature of the HOD

Dr. Shyamala G
Professor & HOD

Examiners

	Name of Examiner	Signature
1		
2		

Academic Year 2026–2027

Acknowledgement

We thank the Principal of B.M.S. College of Engineering and the management of B.M.S. Educational Trust (BMSET) for providing the academic environment and facilities for this work.

We express our gratitude to **Dr. Shyamala G, Professor & Head of the Department, Computer Science and Engineering (Data Science)**, for departmental support. We thank our guide **Manoj Kumar S, Assistant Professor**, for guidance in the Bio-Inspired Algorithms course and for encouraging clear connections between algorithm design, implementation, and experimental evidence.

We also thank the department faculty, the maintainers of the open-source scientific software used in this project, and the researchers who made PlantVillage and the cited research openly accessible. Their contributions make transparent and reproducible student research possible.

The team takes responsibility for the implementation, the integrity of the reported experimental evidence, and the limitations stated in this report. No signature is reproduced electronically; the certificate provides spaces for the required academic sign-off.

Aryanraj S Singh
Richik Chaudhuri
Suraj B
Sushant Deo
Team 10

Abstract

Plant leaf classification is a useful test case for studying how nature-inspired optimization can improve a deep-learning training recipe. This project builds a reproducible system for tomato disease classification using ImageNet-pretrained EfficientNet-B0 and Differential Evolution (DE). The backbone family, dataset, baseline, and evaluation protocol are fixed before model selection. A normalized chromosome controls learning rate, weight decay, dropout, classifier width, fine-tuning depth, label smoothing, and augmentation strength. DE/rand/1/bin is implemented from scratch with distinct donors, parent-to-bound repair, forced binomial crossover, greedy selection, and resumable state.

The cleaned dataset contains 18,146 images, with 12,701, 2,723, and 2,722 assigned to training, validation, and held-out testing. Group-aware splitting reduces leakage from similar images. Candidate fitness is validation macro-F1 at proxy fidelity; final configurations are selected after higher-fidelity reranking and trained over the same repeated-seed protocol. Random search provides an equal-budget control. Evaluation includes per-class metrics, bootstrap intervals, paired classifier comparison, calibration, image-corruption stress tests, and Grad-CAM. Separate analytic benchmarks assess the optimizer independently of neural training.

The current build reports completed data preparation and optimizer benchmarks; final neural-network results remain explicitly pending. No anticipated accuracy is substituted for a measured result.

An offline interactive demo connects diagnosis, model comparison, and visible DE evolution to the underlying evidence. The system emphasizes auditability and honest interpretation over unsupported claims of universal superiority.

Contents

Acknowledgement **ii**

Abstract **iii**

1 Introduction **1**

2 Literature Review **2**

3 Theoretical Background **4**

4 Nature-Inspired Algorithms: Detailed Study **6**

5 Methodology and Implementation **9**

6 Experimental Setup and Test Functions **13**

7 Results and Discussion **16**

8 Comparative Performance Analysis **24**

9 Applications and Case Studies **26**

10 Conclusion **27**

11 Future Work **28**

12 References **29**

13 GitHub Repo Link **32**

List of Figures

4.1 A teaching taxonomy. BBO is biogeography-inspired; categories overlap and do not rank algorithm quality. 6
4.2 Mutation, bound repair, and crossover using a recorded operator example. 7

5.1 System pipeline 9
5.2 EfficientNet-B0 architecture and evolved decisions 11

6.1 Two-dimensional benchmark landscapes 15

7.1 Benchmark convergence across all runs 17
7.2 Distribution of final benchmark errors 17
7.3 DE population snapshots on Rastrigin 18
7.4 DE proxy-fitness progression 19
7.5 Population diversity and replacement success 19
7.6 Candidate fitness projected onto two genes 20
7.7 Parallel coordinates of candidate recipes 20
7.8 Exploratory gene–fitness associations 21
7.9 Final-training learning curves 22

8.1 Equal-budget DE versus random search 24

List of Tables

2.1 Published studies: external results on their own protocols. These are not measurements from this project or a directly comparable leaderboard. . . 3

5.1 Observed class counts after cleaning and group-aware splitting, generated from the saved data summary. 10
5.2 Model block shapes and parameter counts measured from the registered baseline. 11
5.3 Pre-registered baseline. These are fixed design settings, not fitted results. . 11
5.4 Normalized mixed-variable search space. Integer coordinates use half-up rounding after scaling. 12

6.1 Measured environment and configured DE search. Initialization counts toward the candidate budget. 13
6.2 Fixed neural-network training and evaluation protocol. 14
6.3 Benchmark domains and minimizing coordinates, fixed by the implementation. 14

7.1 Final benchmark error: mean and sample standard deviation across all recorded seeds. Smaller is better. 16
7.2 Registered baseline and validation-selected final recipes. 21
7.3 Final held-out metrics, reported as mean and sample standard deviation over the registered training seeds. 21
7.4 Seed-zero bootstrap intervals. These are separate from variation across training seeds. 22
7.5 Seed-zero calibration and mean corruption performance, generated from final evaluation. 22

8.1 Final benchmark errors: median and best over all recorded runs. 25
8.2 Exploratory two-sided rank-sum tests; negative z favors DE. 25
8.3 Measured training, model, and inference cost. 25

13.1 Repository map and reproducibility entry points. 32

CHAPTER 1

Introduction

1.1 Background and problem

Crop pathogens and pests impose substantial losses, with effects that vary by crop and geography [1]. Official Indian horticulture statistics establish tomato as a substantial production crop [2]. Leaf imagery therefore offers a useful application for accessible visual screening. PlantVillage studies demonstrate classification on curated images [3, 4]; they also motivate careful separation of benchmark performance from field reliability.

This project asks whether Differential Evolution (DE) can improve an ImageNet-pretrained EfficientNet-B0 tomato classifier by selecting its training recipe. The objective is validation macro-F1 over a mixed space of learning rate, regularization, head width, fine-tuning depth, and augmentation. Backpropagation fits each candidate's weights; DE selects the recipe. The dataset, backbone family, and optimizer topic remain fixed.

1.2 Measurable objectives

1. Prepare documented class mappings, cleaning records, and group-aware stratified splits.
2. Implement and test DE donor selection, mutation, repair, crossover, and selection.
3. Compare a registered baseline, DE, and equal-budget random search using validation-only selection.
4. Evaluate repeated seeds, per-class performance, uncertainty, robustness, and calibration.
5. Deliver an offline demonstration and reproducible report, slides, code, and evidence.

1.3 Contribution, scope, and organization

The contribution is an auditable chain from candidate vector to checkpoint and reported result. Established methods are combined and evaluated; no claim of a new optimizer or universal superiority is made. Curated-dataset accuracy does not establish reliability for arbitrary crops, unknown diseases, or field photographs.

Chapters 2–4 develop the literature and algorithms; Chapters 5–6 specify implementation and evaluation. Chapters 7–8 analyze the evidence. Applications, conclusions, future work, verified references, and repository instructions complete the report.

CHAPTER 2

Literature Review

2.1 Image-based plant disease classification

Hughes and Salathé released curated plant-health imagery to support machine-learning diagnosis research [3]. Mohanty et al. evaluated deep learning on PlantVillage and also identified the challenge of applying models beyond controlled image conditions [4]. Ferentinos studied a broader multi-crop collection and multiple CNN architectures [5]. Too et al. compared fine-tuned architectures and reported strong DenseNet performance [6]. These papers establish feasibility, but their metrics belong to their own label spaces, splits, and training protocols.

Kamilaris and Prenafeta-Boldú situate image recognition within broader agricultural deep-learning applications [7]. PlantDoc explicitly addresses non-laboratory data [8]. Gui et al. evaluate field recognition separately from PlantVillage performance and investigate background and feature-learning changes [9]. This motivates treating domain shift as a primary limitation rather than using curated test accuracy as a proxy for field readiness.

2.2 Transfer learning and efficient architectures

EfficientNet studies coordinated scaling of depth, width, and resolution [10]. This project uses the fixed B0 architecture and studies its training/head choices, rather than claiming to reproduce the paper's architecture search. MobileNetV2 and MobileNetV3 are relevant examples of efficient design and hardware-aware model development [11, 12]; they remain contextual alternatives outside the fixed submitted backbone.

Yosinski et al. show that transferability depends on the level of a feature and the relation between source and target tasks [13]. Fine-tuning depth is consequently a meaningful decision variable. Freezing the entire backbone is a defensible baseline, while unfreezing more layers may improve adaptation or increase overfitting and computation. The direction must be measured on validation data.

2.3 Hyperparameter and evolutionary search

Grid search enumerates a Cartesian product of settings; random search samples configurations independently. Bayesian optimization models an expensive objective to choose informative trials [14]. Hyperband emphasizes adaptive resource allocation and early stopping [15]. These alternatives clarify the trade-off between search intelligence and evaluation cost. A random-search control is essential: it tests whether population feedback contributes beyond a comparable number of independently sampled recipes.

Evolutionary search has credible precedents in model design. Real et al. study regularized evolution for image-classifier architectures [16]. Wang et al. develop variable-length CNN search with PSO and a separate hybrid DE approach [17, 18]. DEHB combines DE with Hyperband for HPO [19]. Their search spaces and budgets differ from this project's seven-gene recipe. They justify investigation, not a predicted advantage on tomato images.

Table 2.1: Published studies: external results on their own protocols. These are not measurements from this project or a directly comparable leaderboard.

Study	Dataset / model	Search or comparison	Accuracy
Mohanty et al. [4]	PlantVillage, 38 classes; AlexNet/GoogLeNet	Training/model variants	99.35%
Ferentinos [5]	58 categories; CNNs	CNN architectures	99.53%
Too et al. [6]	PlantVillage, 38 classes; DenseNet	Fine-tuned architectures	99.75%
Gui et al. [9]	Field-PV; adapted CNN	Background/features	72.03%

2.4 Research gap

The useful gap for this project is an auditable comparison under a fixed small-project protocol: grouped data splits, a pre-registered frozen-backbone baseline, explicit mixed-variable decoding, equal candidate budgets, higher-fidelity reranking, and held-out repeated-seed evaluation. The work connects algorithm-level tests to application-level evidence and an offline demonstration. Its scientific value does not require DE to win every comparison; a neutral result can identify the limits of a small-budget optimizer or a noisy proxy.

CHAPTER 3

Theoretical Background

3.1 Representation and transfer learning

Convolutions share local filters; nonlinear activations and pooling build hierarchical representations. EfficientNet-B0 combines mobile inverted bottlenecks (MBConv), depthwise convolution, and squeeze-and-excitation channel attention. Compound scaling coordinates depth, width, and resolution, conventionally $d = \alpha^\phi$, $w = \beta^\phi$, $r = \gamma^\phi$ with $\alpha\beta^2\gamma^2 \approx 2$ [10]. This project retains B0.

The pooled representation enters dropout, an optional hidden linear layer with SiLU and dropout, and ten output logits. Hidden width zero removes the hidden layer. Softmax gives $p_c = \exp(z_c)/\sum_j \exp(z_j)$ within the known classes; it does not detect an unknown crop. Frozen blocks preserve both parameters and BatchNorm statistics. Unfreezing later blocks permits adaptation, consistent with the dependence of transferability on feature level [13].

3.2 Training and regularization

Cross-entropy minimizes $\mathcal{L} = -N^{-1}\sum_i \sum_c q_{ic} \log p_{ic}$. Dropout suppresses activations; weight decay penalizes parameter magnitude; label smoothing replaces one-hot targets by $q_{ic} = (1-\epsilon)y_{ic} + \epsilon/C$. Augmentation changes observed appearance. The registered baseline uses Adam with zero weight decay; searched recipes use AdamW's decoupled decay. This difference belongs to the overall recipe comparison.

A linear warmup precedes cosine learning-rate decay. Gradient clipping limits gradient norm. Early stopping monitors validation macro-F1 and restores its best checkpoint. DE fitness is this validation metric after the allotted training, rather than training loss.

3.3 Classification and ranking metrics

For class c, using counts TP_c, FP_c, and FN_c,

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F_{1,c} = \frac{2P_cR_c}{P_c + R_c}, \tag{3.1}$$

$$F_{1,\text{macro}} = \frac{1}{C}\sum_c F_{1,c}, \qquad \text{Accuracy} = \frac{1}{N}\sum_i \mathbf{1}[\hat{y}_i = y_i]. \tag{3.2}$$

Undefined ratios are assigned zero consistently. Macro averages give classes equal weight; weighted averages use class support. One-versus-rest ROC varies a class-score threshold, plotting true-positive rate against false-positive rate; $\text{AUC} = \int_0^1 \text{TPR}(u)\,du$. Macro AUC averages classwise areas. Top-three accuracy counts labels among the three largest scores.

3.4 Uncertainty and paired comparisons

Bootstrap resampling recomputes metrics on sampled observations [20]. Image-level intervals assume sufficiently independent observations; related leaf views limit that assumption. Sampling intervals and variation across training seeds answer different questions.

For paired classifiers, let b, c count opposite-direction correctness disagreements. The continuity-corrected McNemar statistic is $(|b-c|-1)^2/(b+c)$ [21]. Our implementation instead uses the exact two-sided binomial test conditional on $b+c$, with $p=1$ when no disagreements occur. Effect size remains necessary even when a test is significant.

3.5 Calibration, explanations, and robustness

Expected calibration error compares correctness with confidence in bins B_m [22]:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|. \tag{3.3}$$

Its value depends on binning and does not equal accuracy. Grad-CAM rectifies a gradient-weighted sum of feature maps [23]; it is a coarse explanation, not a lesion annotation. Synthetic corruptions probe prescribed degradations [24]; neither technique establishes field reliability alone.

CHAPTER 4

Nature-Inspired Algorithms: Detailed Study

4.1 Classification and shared search principles

Metaheuristics guide search without a useful analytic optimum. Population methods maintain several candidates; trajectory methods modify one state. Exploration visits new regions, while exploitation refines promising ones.

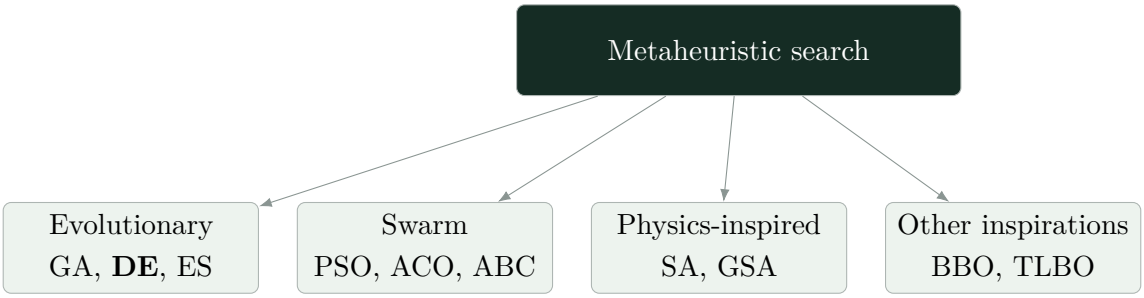


Figure 4.1: A teaching taxonomy. BBO is biogeography-inspired; categories overlap and do not rank algorithm quality.

GA combines selection and variation [25]; PSO maintains individual and social memories [26]. Ant System uses pheromone feedback [27]; simulated annealing uses temperature-controlled acceptance [28].

4.2 Representative update mechanisms

A real-coded GA recombines selected parents, for example $\mathbf{u} = \alpha\mathbf{x}_a + (1-\alpha)\mathbf{x}_b$, then mutates and selects survivors. Our comparator uses tournament selection, blend crossover, Gaussian mutation, and elitism.

For PSO, particle velocity and position are updated as

$$\mathbf{v}_i^{t+1} = w\mathbf{v}_i^t + c_1\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^t) + c_2\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^t), \tag{4.1}$$

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1}, \tag{4.2}$$

where $\mathbf{p}_i$ is the particle's best position and $\mathbf{g}$ the population best. In an ant system, transition preference is proportional to pheromone τ_{ij}^{α} times heuristic desirability η_{ij}^{β}; pheromone is evaporated and reinforced by completed solutions [27]. Simulated annealing accepts an uphill energy change ΔE with probability $\exp(-\Delta E/T)$, allowing controlled escape from local basins [28].

Firefly-style movement combines attraction and random perturbation, often expressed as $\mathbf{x}_i \leftarrow \mathbf{x}_i + \beta_0 e^{-\gamma r_{ij}^2}(\mathbf{x}_j - \mathbf{x}_i) + \alpha\boldsymbol{\epsilon}$ [29]. BBO models immigration and emigration between candidate habitats; a simple linear rate model uses $\lambda(s) = I(1 - s/S)$ and $\mu(s) = Es/S$

for species count s [30]. ABC uses employed/onlooker/scout search roles [31]; GWO models social hierarchy and hunting-inspired position updates [32]. These are background methods, not additional CNN experiments.

4.3 Differential Evolution in depth

DE maintains NP vectors in a normalized domain. In DE/rand/1, mutation for target i is

$$\mathbf{v}_i = \mathbf{x}_{r_1} + F(\mathbf{x}_{r_2} - \mathbf{x}_{r_3}), \qquad r_1, r_2, r_3 \neq i, \tag{4.3}$$

with distinct donor indices. Difference vectors supply directions and scales informed by the current population [33, 34]. The implementation freezes the donor population within each generation, so earlier evaluations do not change later donors in the same generation.

For a coordinate outside $[0, 1]$, the repair rule draws a feasible value between the parent and the violated bound:

$$v_{ij} \leftarrow x_{ij} + r(b - x_{ij}), \quad r \sim U(0, 1), \quad b \in \{0, 1\}. \tag{4.4}$$

Valid coordinates remain unchanged. This repair avoids repeated clipped boundary values but does not preserve uniform sampling.

Binomial crossover forms

$$u_{ij} = \begin{cases} v_{ij}, & r_j < CR \text{ or } j = j_{\text{rand}}, \\ x_{ij}, & \text{otherwise.} \end{cases} \tag{4.5}$$

The forced coordinate ensures mutant contribution. Maximizing selection is

$$\mathbf{x}_i^{g+1} = \begin{cases} \mathbf{u}_i, & f(\mathbf{u}_i) \geq f(\mathbf{x}_i^g), \\ \mathbf{x}_i^g, & \text{otherwise.} \end{cases} \tag{4.6}$$

Best stored fitness is nondecreasing, although noisy fitness can misrepresent true quality. Difference-vector mutation is rotation-equivariant; coordinate-wise crossover prevents the whole algorithm from being rotation-invariant.

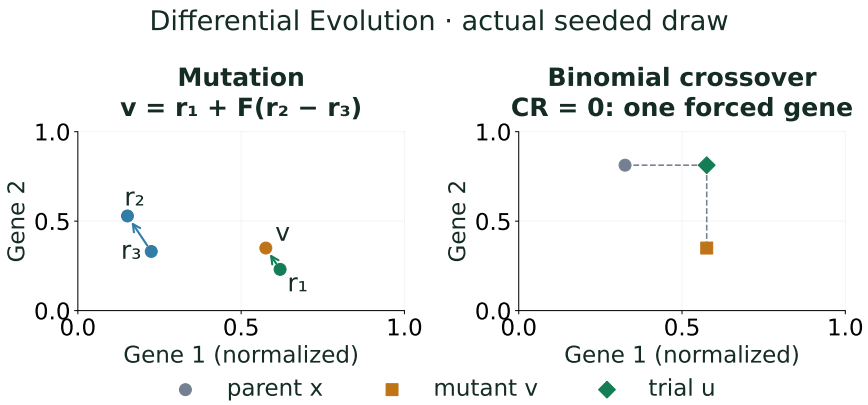


Figure 4.2: Mutation, bound repair, and crossover using a recorded operator example.

4.4 Pseudocode and complexity

Algorithm 1 DE/rand/1/bin with deferred generation donors

Require: Objective f, dimension D, population NP, generations G, F, CR

Initialize X with Latin-hypercube samples in $[0, 1]^D$
Evaluate all initial candidates and persist state
for $g = 1, \ldots, G$ **do**
    $S \leftarrow X$ ▷ Frozen donor population
    for $i = 1, \ldots, NP$ **do**
        Choose three distinct donor indices excluding i
        $v \leftarrow S_{r_1} + F(S_{r_2} - S_{r_3})$
        Repair violating coordinates between X_i and the bound
        Form u using binomial crossover and forced j_{rand}
        Evaluate $f(u)$ using the same fitness protocol
        if $f(u) \geq f(X_i)$ **then**
            $X_i \leftarrow u$
        end if
        Atomically persist population, scores, and RNG state
    end for
    Record fitness distribution, diversity, and success rate
end for
return best recorded vector and full history

The budget is $NP(G + 1)$ including initialization; vector arithmetic is $O(GNPD)$, while training dominates time. Mean pairwise distance measures diversity, and replacement fraction measures success rate; both help identify stagnation.

4.5 Variants and parameter control

DE/best/1 uses the current best as its mutation base; DE/rand/2 uses two difference vectors; current-to-best strategies add attraction toward a leading vector. Such changes modify exploration and exploitation [35]. Increasing F expands differential steps, while CR changes coordinate mixing; their useful values depend on the landscape and available budget.

jDE retains successful individual control values [36]; SHADE uses success histories and L-SHADE reduces population size [37, 38]. Only classic DE optimizes this CNN; adaptive variants are benchmark comparators.

Why DE here. Seven normalized genes support derivative-free mixed search with few controls and independent candidate evaluations. Noisy fitness, repeated recipes, and small populations limit coverage; equal-budget evidence determines usefulness [39].

CHAPTER 5

Methodology and Implementation

5.1 End-to-end evidence flow

The pipeline separates preparation, model selection, final training, and final evaluation. Numbered scripts expose each stage, while small shared modules implement data handling, the model, training, and optimizers. The offline demo consumes saved artifacts and project checkpoints. It does not manufacture metrics when those files are absent.

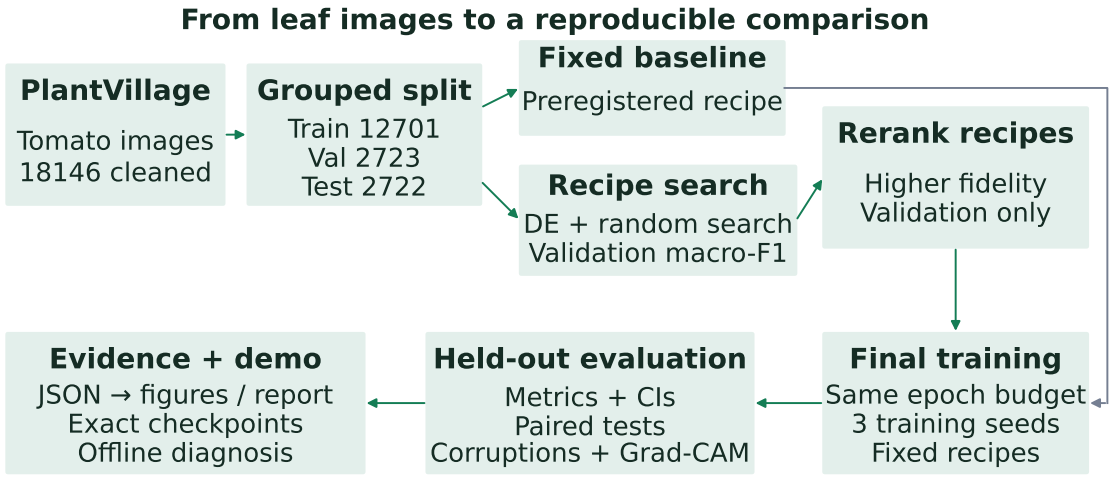


Figure 5.1: Implemented evidence flow from data preparation to model selection, final evaluation, and the offline demo.

5.2 Data source, cleaning, and splitting

The color tomato folders are retrieved from the PlantVillage GitHub mirror. Folder names are sorted alphabetically to define the fixed class mapping. Each image is verified with Pillow, oriented consistently using EXIF information, and converted to RGB. A SHA-256 digest over image dimensions and decoded pixels identifies exact duplicate content. Perceptual hashes identify near-neighbor relationships; connected groups include transitive relationships and are constructed across classes rather than independently within each class. These visual-similarity groups are not verified plant or capture-session identities and do not establish cross-farm independence.

The observed input contains 18,160 images. Cleaning removes 14 exact duplicates and 0 corrupt images, leaving 18,146. Group-aware assignment targets a 70/15/15 class-stratified train/validation/test split using a fixed split seed. Because groups are indivisible, exact proportions may differ slightly. Committed CSV manifests retain the relative

path, class, perceptual hash, content digest, and group identifier. Reusing an existing manifest verifies image digests rather than silently constructing a new split.

Table 5.1: Observed class counts after cleaning and group-aware splitting, generated from the saved data summary.

Class	Original	Train	Validation	Test
Bacterial spot	2,127	1,489	319	319
Early blight	1,000	700	150	150
Late blight	1,909	1,331	285	285
Leaf Mold	952	666	143	143
Septoria leaf spot	1,771	1,239	266	266
Spider mites	1,676	1,174	251	251
Target Spot	1,404	982	211	211
Yellow Leaf Curl Virus	5,357	3,750	804	803
mosaic virus	373	261	56	56
healthy	1,591	1,109	238	238
Total	18,160	12,701	2,723	2,722

5.3 Transforms and caching

The image cache resizes the shorter side to 256 pixels and saves high-quality JPEG copies. Training uses random resized crops and horizontal/vertical flips, with RandAugment added when the augmentation gene is nonzero. The baseline uses the common crop-and-flip pipeline without RandAugment. Validation uses deterministic resize and center crop. ImageNet normalization is applied consistently. The cache accelerates file decoding without changing split membership or the selected labels.

5.4 Model and registered baseline

EfficientNet-B0 uses ImageNet-pretrained weights and the task-specific head described in Chapter 3. Fine-tuning depth k selects the last k of the nine feature children. For $k = 0$, only the classifier is trainable; for $k = 9$, all feature blocks are trainable. The registered baseline's measured trainable parameter count is 330,506.

5.5 Chromosome and candidate fitness

Every candidate trains a fresh pretrained model on the same fixed stratified training subset and training seed at proxy resolution. The entire validation split supplies macro-F1 fitness. Decoded floating values are rounded consistently for both training and cache identity; a cached recipe is never treated as a different training experiment merely because its raw genes differ. Failures due to nonfinite training or memory exhaustion receive the documented zero fitness and retain an error record. Unrelated programming exceptions are allowed to fail visibly.

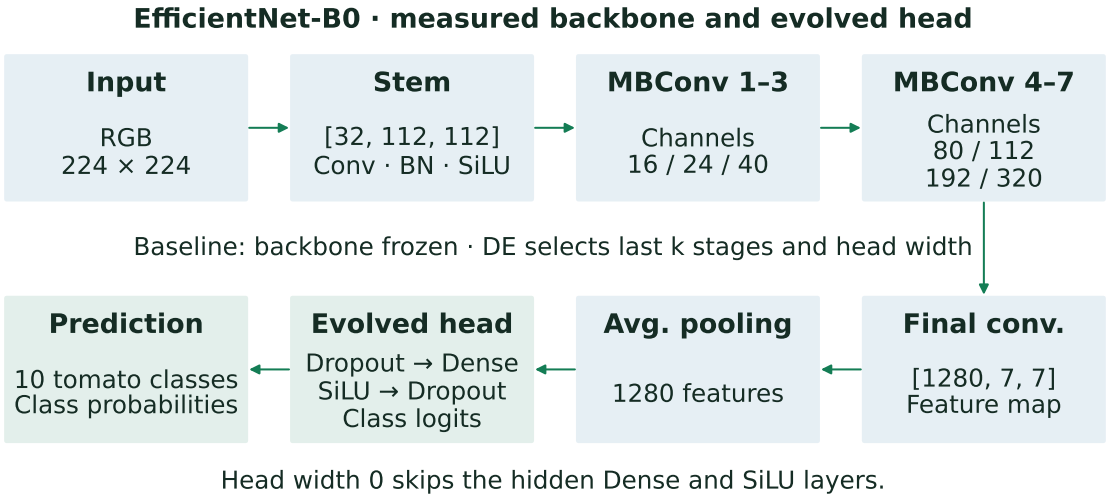


Figure 5.2: Actual backbone/head structure and the decisions controlled by the chromosome.

Table 5.2: Model block shapes and parameter counts measured from the registered baseline.

Block	Output shape	Parameters	Trainable (baseline)
0	1 Œ 32 Œ 112 Œ 112	928	0
1	1 Œ 16 Œ 112 Œ 112	1,448	0
2	1 Œ 24 Œ 56 Œ 56	16,714	0
3	1 Œ 40 Œ 28 Œ 28	46,640	0
4	1 Œ 80 Œ 14 Œ 14	242,930	0
5	1 Œ 112 Œ 14 Œ 14	543,148	0
6	1 Œ 192 Œ 7 Œ 7	2,026,348	0
7	1 Œ 320 Œ 7 Œ 7	717,232	0
8	1 Œ 1280 Œ 7 Œ 7	412,160	0
classifier	1 Œ 10	330,506	330,506

Table 5.3: Pre-registered baseline. These are fixed design settings, not fitted results.

Setting	Value
Backbone	ImageNet-pretrained EfficientNet-B0
Unfrozen feature blocks	0
Classifier hidden units	256
Dropout	0.2
Optimizer / learning rate	Adam / 10^{-3}
Weight decay / label smoothing	0 / 0
Extra RandAugment	Disabled
Batch / final resolution	32 / 224 × 224
Epoch ceiling / patience	15 / 4

Table 5.4: Normalized mixed-variable search space. Integer coordinates use half-up rounding after scaling.

Gene	Decision	Decoded range
x_1	Learning rate	$10^{-4.5+2.5x_1}$
x_2	Weight decay	10^{-6+4x_2}
x_3	Dropout	$0.6x_3$
x_4	Hidden units	Index into $\{0, 128, 256, 512\}$
x_5	Unfrozen blocks	Integer in $\{0, \ldots, 9\}$
x_6	Label smoothing	$0.2x_6$
x_7	RandAugment magnitude	Integer in $\{0, \ldots, 15\}$

The DE population is Latin-hypercube initialized without inserting the baseline. A separately measured baseline at proxy fidelity supplies a comparable reference line. It is not counted as a DE-generated candidate. After search, the top distinct recipes undergo higher-fidelity validation reranking. Random search uses the same fitness protocol, candidate budget, and reranking rule.

5.6 Durability, provenance, and deployment

The search state includes the population, scores, generator state, generation, and next individual. Atomic replacement avoids partially written checkpoints. Durable evaluation events reconcile the CSV after interruptions. Training checkpoints record configuration, seed, resolution, class mapping, protocol fingerprint, and best validation epoch. Protocol mismatches reject unsafe resumption rather than silently mixing experiments.

The local demo reads the seed-zero baseline and DE checkpoints and performs real inference. Grad-CAM is computed from the final convolutional feature output using a documented gradient implementation. Input validation catches unsupported files and excessive image sizes. A missing checkpoint produces an explicit unavailable state. Installation may require downloads; after setup, inference and optimizer animation run locally without external services.

CHAPTER 6

Experimental Setup and Test Functions

6.1 Hardware and software

Synthetic-tensor throughput and finite-loss checks inform tier B (Table 6.1). These planning estimates exclude loading, model construction, and pipeline overhead; they are not wall-clock upper bounds or measured full-pipeline runtimes.

Table 6.1: Measured environment and configured DE search. Initialization counts toward the candidate budget.

Item	Measured / configured value
CPU	Apple M4 Pro
Memory (GiB)	48
Device	mps
Python	3.11.14
PyTorch	2.15.0
Operating system	macOS-26.7.1-arm64-arm-64bit
Compute tier	B

Setting	Value
Proxy resolution	128
Training fraction	20%
Proxy epochs	3
DE population	8
DE generations	8
Budget per search	72
Scale F	0.6
Crossover CR	0.9
Search seed	42

6.2 Neural-network protocol

Table 6.2 fixes training and validation-only selection. Evaluation verifies the baseline registration and frozen protocols before accessing test data; resumption requires identical checkpoints and manifests. Bootstrap intervals and training-seed variation answer separate questions.

Latency measures warmed, synchronized, batch-one forward passes; decoding, Grad-CAM, and rendering add costs. Search logs distinguish timed candidate training from cached reevaluations.

Table 6.2: Fixed neural-network training and evaluation protocol.

Stage	Configured / frozen protocol
Data partition	Train / validation / test: 12,701 / 2,723 / 2,722; grouped split seed 42.
Proxy search	128 px; 20% of training data; 3 epochs; 72 candidate evaluations per method.
Search seeds	Proposal generation: DE 42, random search 43; shared fitness-training seed 42.
Validation reranking	Top 3 distinct candidates per method; 224 px; full training split; 3 epochs; training seed 42.
Final training	224 px; full training split; at most 15 epochs; patience 4; seeds 0, 1, 2.
Shared training controls	Batch size 32; warmup 1 epoch(s); gradient clipping norm 1.
Optimizers	Baseline: Adam; searched recipes: AdamW.
Analytic benchmarks	10 dimensions; population 40 for population methods; 20,000 evaluations per run; 30 independent runs per method and function.

6.3 Analytic test functions

These NumPy objectives span smooth, curved-valley, and multimodal landscapes [40], each with minimum zero. Quantitative runs use ten dimensions; views use two.

$$f_{\text{Sphere}}(\mathbf{x}) = \sum_{i=1}^{D} x_i^2, \tag{6.1}$$

$$f_{\text{Rosenbrock}}(\mathbf{x}) = \sum_{i=1}^{D-1} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right], \tag{6.2}$$

$$f_{\text{Rastrigin}}(\mathbf{x}) = 10D + \sum_{i=1}^{D} \left[x_i^2 - 10\cos(2\pi x_i)\right], \tag{6.3}$$

$$f_{\text{Ackley}}(\mathbf{x}) = -20e^{-0.2\sqrt{D^{-1}\sum_i x_i^2}} - e^{D^{-1}\sum_i \cos(2\pi x_i)} + 20 + e, \tag{6.4}$$

$$f_{\text{Griewank}}(\mathbf{x}) = 1 + \frac{1}{4000}\sum_{i=1}^{D} x_i^2 - \prod_{i=1}^{D} \cos\left(\frac{x_i}{\sqrt{i}}\right). \tag{6.5}$$

Table 6.3: Benchmark domains and minimizing coordinates, fixed by the implementation.

Function	Domain per coordinate	Minimizer
Sphere	$[-5.12, 5.12]$	$x_i = 0$
Rosenbrock	$[-5, 10]$	$x_i = 1$
Rastrigin	$[-5.12, 5.12]$	$x_i = 0$
Ackley	$[-32.768, 32.768]$	$x_i = 0$
Griewank	$[-600, 600]$	$x_i = 0$

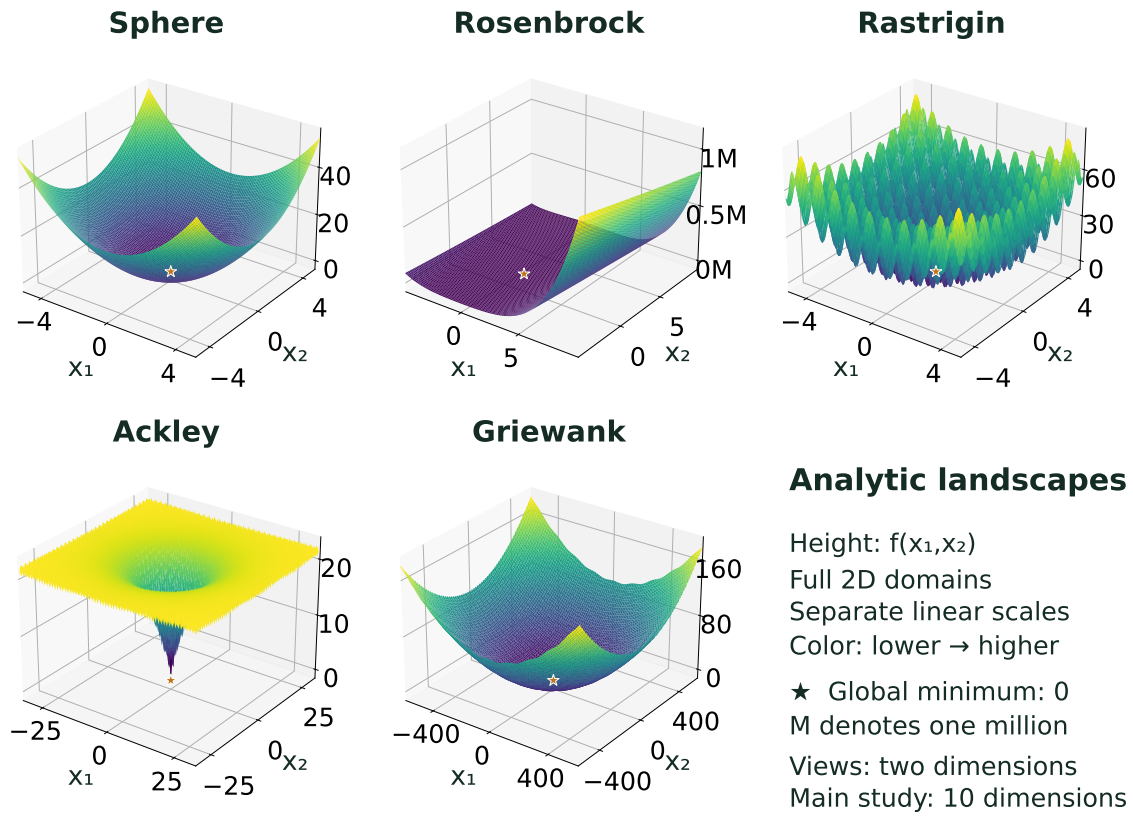


Figure 6.1: Two-dimensional benchmark landscapes. The quantitative study uses ten dimensions.

6.4 Equal-budget optimizer comparison

Each method receives 20,000 evaluations including initialization, across 30 seeds: 750 runs and 15,000,000 calls. The maximizing API negates objectives; reported errors restore minimization values.

Population methods initialize 40 Latin-hypercube candidates; RS samples independently; seeds are 0–29. DE uses $F = 0.6$, $CR = 0.9$. jDE resamples each of $F \sim U(0.1, 1)$ and $CR \sim U(0, 1)$ with probability 0.1, retaining successful controls. GA uses tournament-three selection, BLX-0.5 crossover, Gaussian mutation ($\sigma = 0.1$, coordinate probability $1/D$), and elitism. Global-best PSO uses $w = 0.7298$, $c_1 = c_2 = 1.49618$, initial velocity $U(-0.1, 0.1)$, limit 0.2, and boundary damping -0.5.

Curves show median/interquartile range; tables show mean/sample standard deviation and median/best. Two-sided rank-sum tests are exploratory, unadjusted, and not tie-corrected despite ties; approximate p-values are not confirmatory (Tables 8.1 and 8.2).

6.5 Corruption protocol

Seed-zero models receive identical test images under Gaussian blur, Gaussian noise, brightness/contrast reduction, JPEG compression, and low-light gamma, each at three severities. Mean corruption accuracy equally weights these deterministic conditions, not farm conditions.

CHAPTER 7

Results and Discussion

Experiment status. Final neural-network retraining and held-out evaluation are not complete in this build. Missing values are explicitly marked pending. They must not be presented as completed results.

7.1 Optimizer validation on analytic functions

The completed benchmark study evaluates all methods under the same function-call budget. The median DE Sphere error is 2.59×10^{-15}, demonstrating convergence on the smooth ten-dimensional test. This is evidence about the optimizer implementation; it does not establish tomato-classification accuracy.

Table 7.1: Final benchmark error: mean and sample standard deviation across all recorded seeds. Smaller is better.

Function	DE	jDE	GA	PSO	Random Search
Sphere	4.786×10^{-15} $\pm$ 6.542×10^{-15}	3.113×10^{-23} $\pm$ 5.092×10^{-23}	0.001 121 $\pm$ 0.000 499 4	1.595×10^{-25} $\pm$ 3.598×10^{-25}	11.48 $\pm$ 2.71
Rosenbrock	0.7347 $\pm$ 0.8759	3.243 $\pm$ 0.91	9.206 $\pm$ 12.13	3.118 $\pm$ 2.094	8972 $\pm$ 3475
Rastrigin	28.91 $\pm$ 6.388	6.124×10^{-11} $\pm$ 2.315×10^{-10}	3.695 $\pm$ 1.916	7.429 $\pm$ 4.071	64.11 $\pm$ 7.415
Ackley	6.847×10^{-7} $\pm$ 5.662×10^{-7}	3.092×10^{-11} $\pm$ 2.28×10^{-11}	0.4568 $\pm$ 0.1393	3.517×10^{-12} $\pm$ 3.736×10^{-12}	16.46 $\pm$ 0.9384
Griewank	0.4422 $\pm$ 0.097 86	0.001 879 $\pm$ 0.004 593	0.6296 $\pm$ 0.1149	0.084 29 $\pm$ 0.033 51	40.42 $\pm$ 9.303

The landscapes do not produce a universal winner. Classic DE is effective on the smooth Sphere landscape, while the recorded multimodal results distinguish it from adaptive jDE and other methods. The full table, rather than one selected function, is the appropriate evidence. Values near floating-point precision should be interpreted as numerical convergence rather than practically meaningful decimal-level superiority.

7.2 Search trajectory and population behavior

The saved DE log contains 72 candidate evaluations. Best/mean fitness, population variability, diversity, and replacement success rate show different aspects of search. The baseline reference is evaluated at the same proxy fidelity. A final full-resolution score must not be placed on a proxy convergence curve as if the conditions matched.

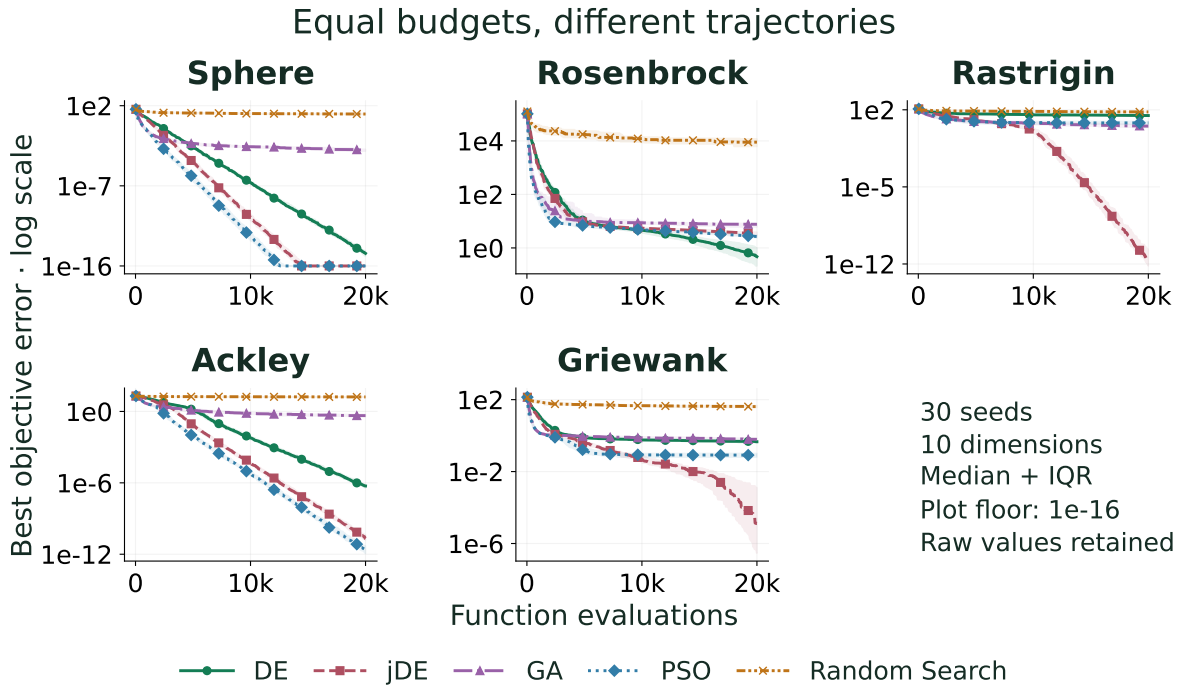


Figure 7.1: Median best-so-far benchmark error with interquartile bands against objective evaluations.

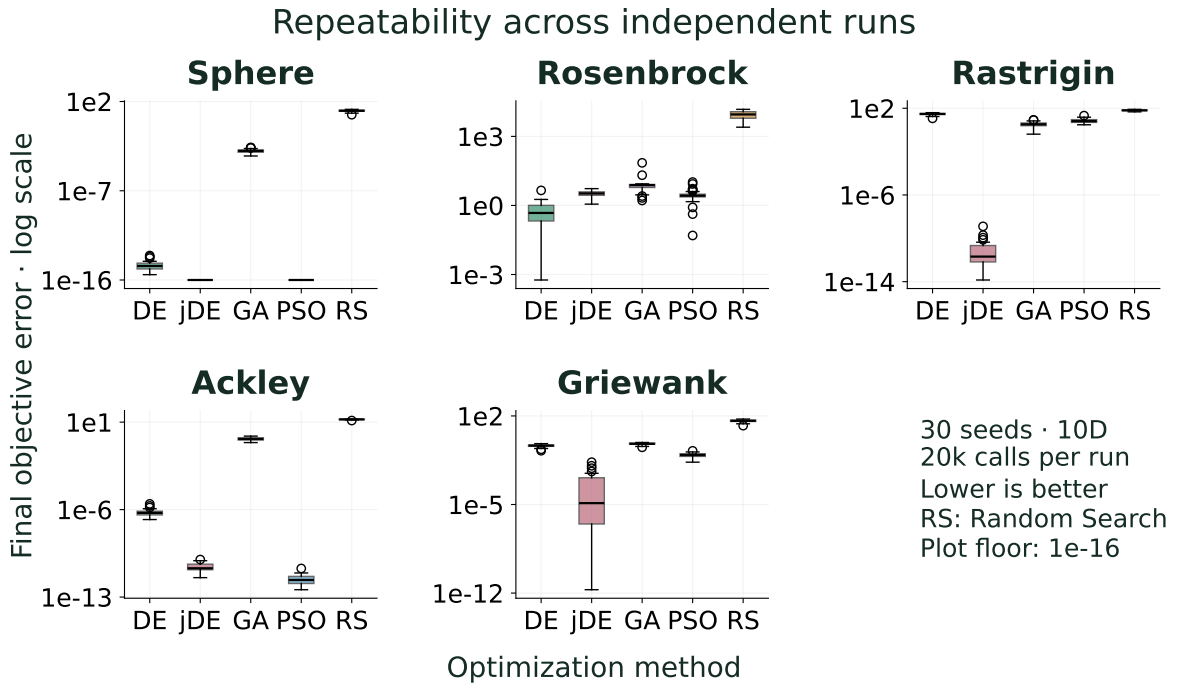


Figure 7.2: Distribution of final benchmark errors over repeated runs. Methods retain consistent labels and colors.

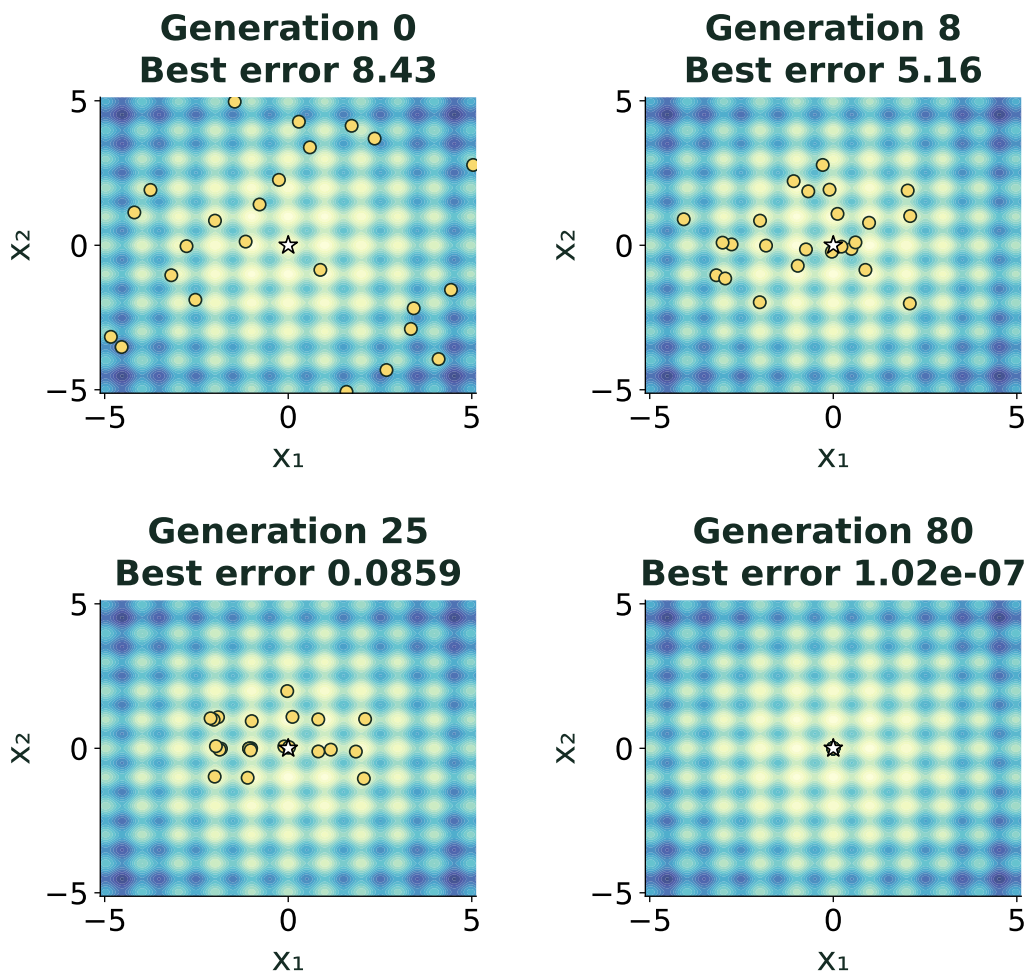


Figure 7.3: Recorded two-dimensional DE population snapshots. This visualization explains the search; the quantitative benchmark uses the separately specified main dimension.

7.3 Selected configuration and learning behavior

Selection follows higher-fidelity reranking on validation data. The generated configuration table compares the registered baseline with the chosen DE and random-search recipes. Training curves track optimization and validation behavior under the final protocol, including early stopping. Mean and standard deviation retain all seeds by truncating each method to their shared epochs. Cross-recipe raw-loss differences also reflect different label-smoothing settings. A training/validation gap or noisy proxy score should be discussed rather than concealed by presenting only the best epoch.

7.4 Held-out classification comparison

Held-out classification values are not yet available in this build. The tables and figures will be populated only after the final protocol completes; analytic benchmark performance is not substituted for neural-network evidence.

Figure awaiting its measured source artifacts: `fig_before_after_bars`.

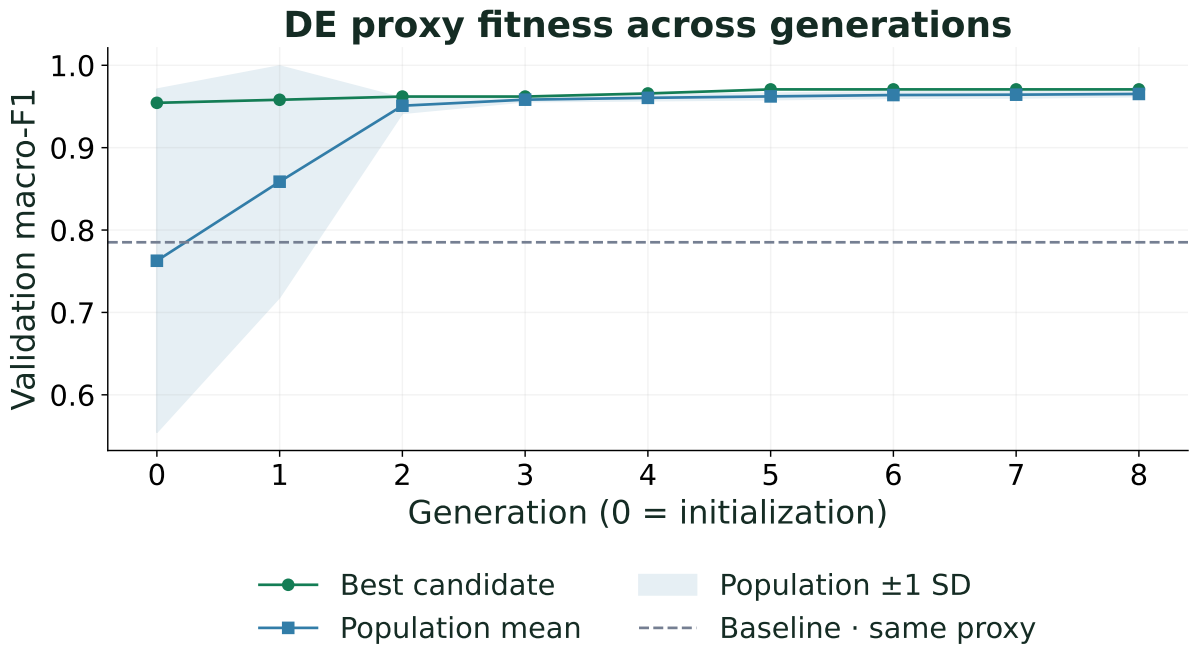


Figure 7.4: DE validation-fitness progression; the spread describes the population, not a confidence interval over independent searches.

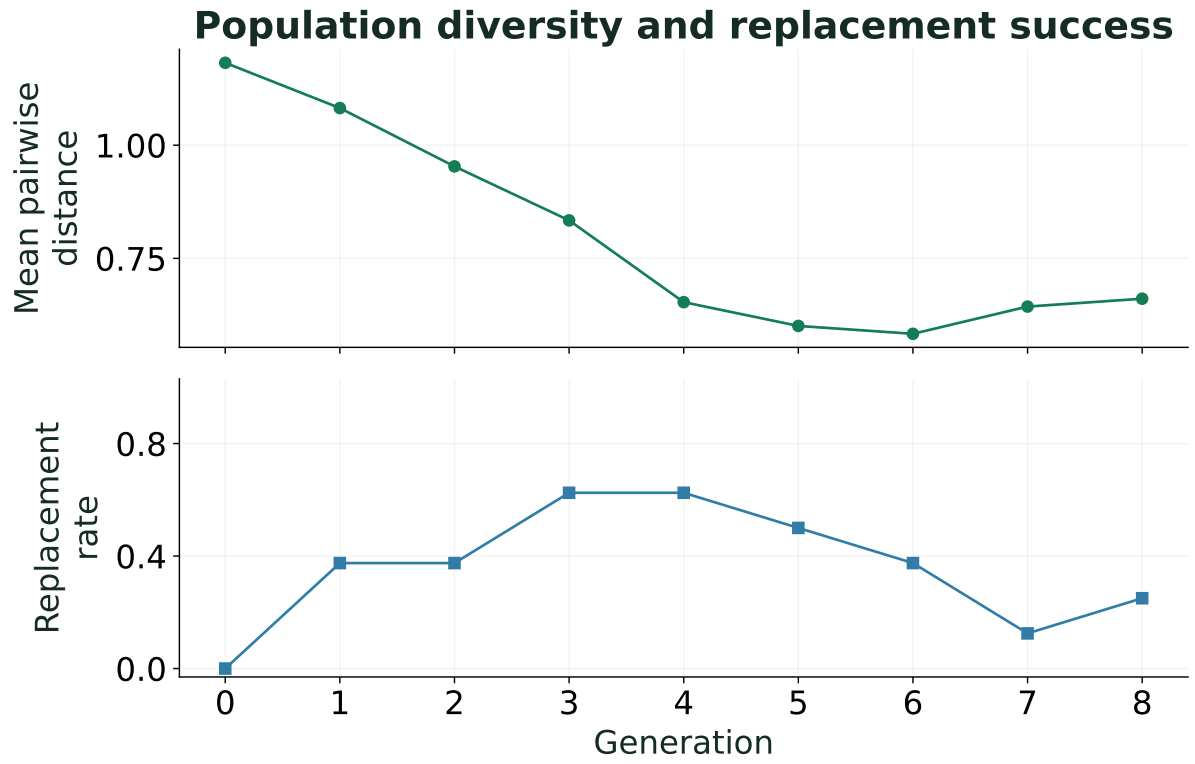


Figure 7.5: Population diversity and successful replacement rate across generations.

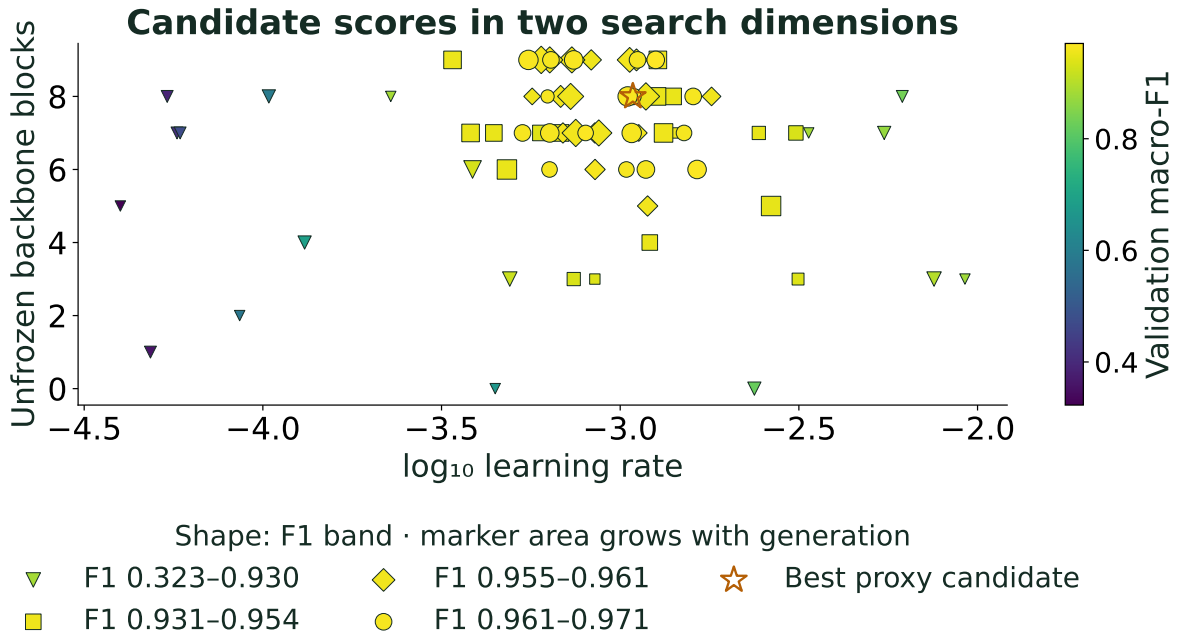


Figure 7.6: Observed candidate fitness projected onto learning rate and fine-tuning depth.

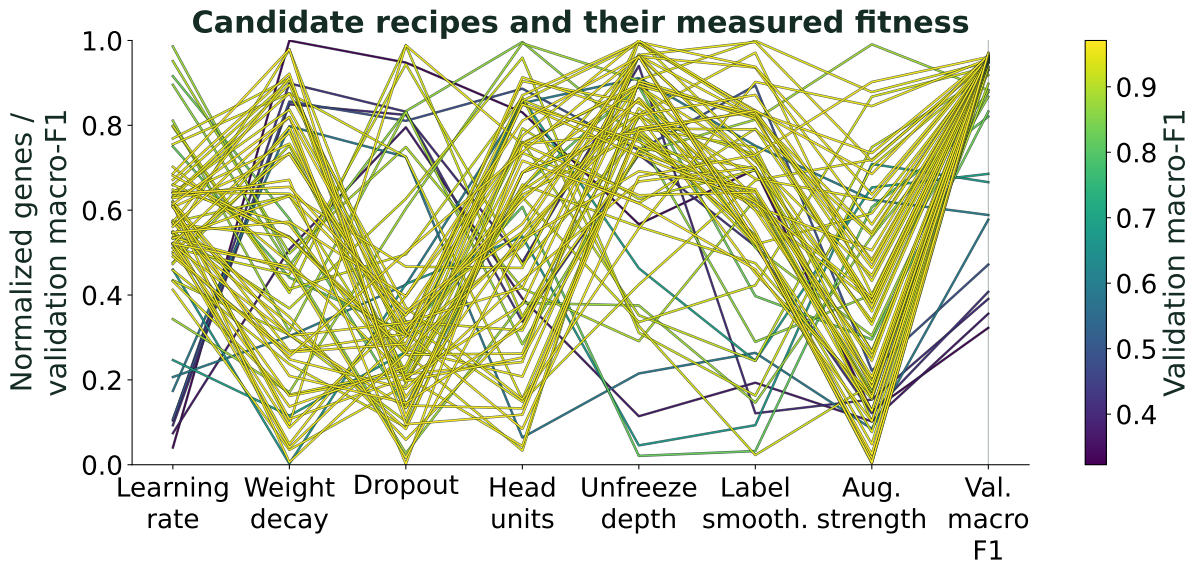


Figure 7.7: All candidate recipes in normalized coordinates; discrete decoding can merge different vectors.

Figure awaiting its measured source artifacts: `fig_confusion_side_by_side`.

Figure awaiting its measured source artifacts: `fig_per_class_f1`.

Figure awaiting its measured source artifacts: `fig_roc`.

Confusion patterns reveal whether gains are distributed or concentrated in a few classes. Similar symptoms can cause confusion, but attributing an error to a specific biological mechanism requires inspection and expert knowledge. Class imbalance makes per-class

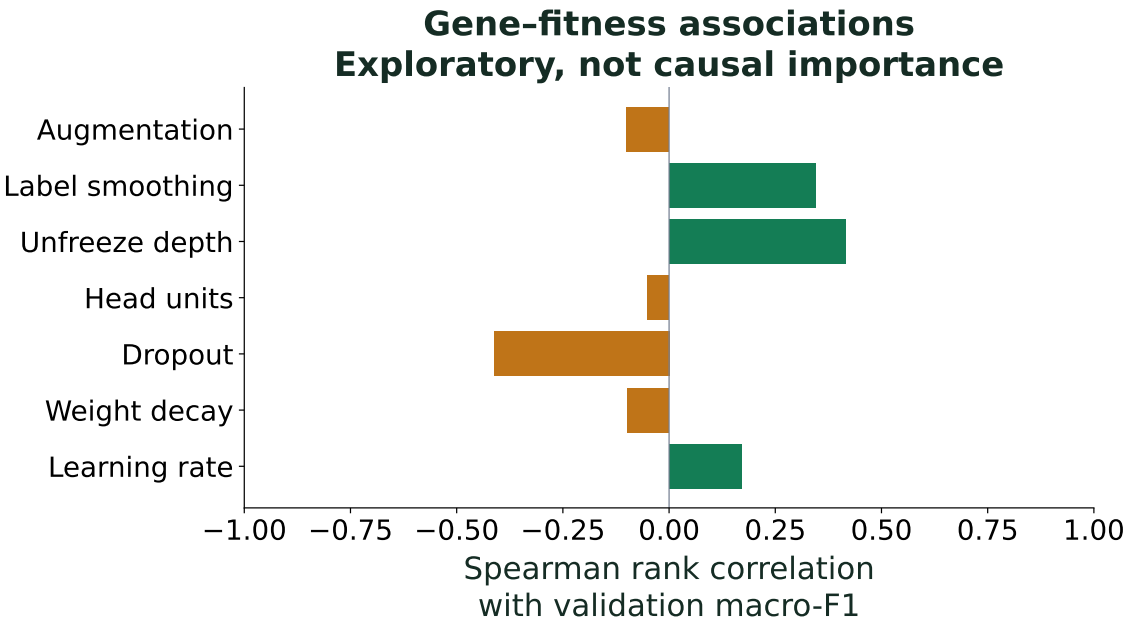


Figure 7.8: Exploratory gene–fitness associations; these are not causal effects.

Table 7.2: Registered baseline and validation-selected final recipes.

Parameter	Baseline	DE	Random search
Learning rate	0.001	0.001 257	0.000 656 3
Weight decay	0	0.000 416 8	2.568×10^{-6}
Dropout	0.2	0.2167341861	0.0293420375
Hidden units	256	256	512
Unfrozen blocks	0	9	6
Label smoothing	0.0	0.1056838563	0.0987632542
Augmentation magnitude	0	0	11

Table 7.3: Final held-out metrics, reported as mean and sample standard deviation over the registered training seeds.

Model	Accuracy (%)	Precision (%)	Recall (%)	Macro-F1 (%)	ROC-AUC (%)
Pending: required experiment evidence is not available.					

recall and macro-F1 particularly relevant. A near-ceiling accuracy alone is insufficient evidence that the system handles rare classes well.

7.5 Uncertainty and paired evidence

Confidence intervals and the paired test remain pending the frozen-checkpoint evaluation.

7.6 Robustness, confidence, and qualitative inspection

Figure awaiting its measured source artifacts: `fig_robustness`.

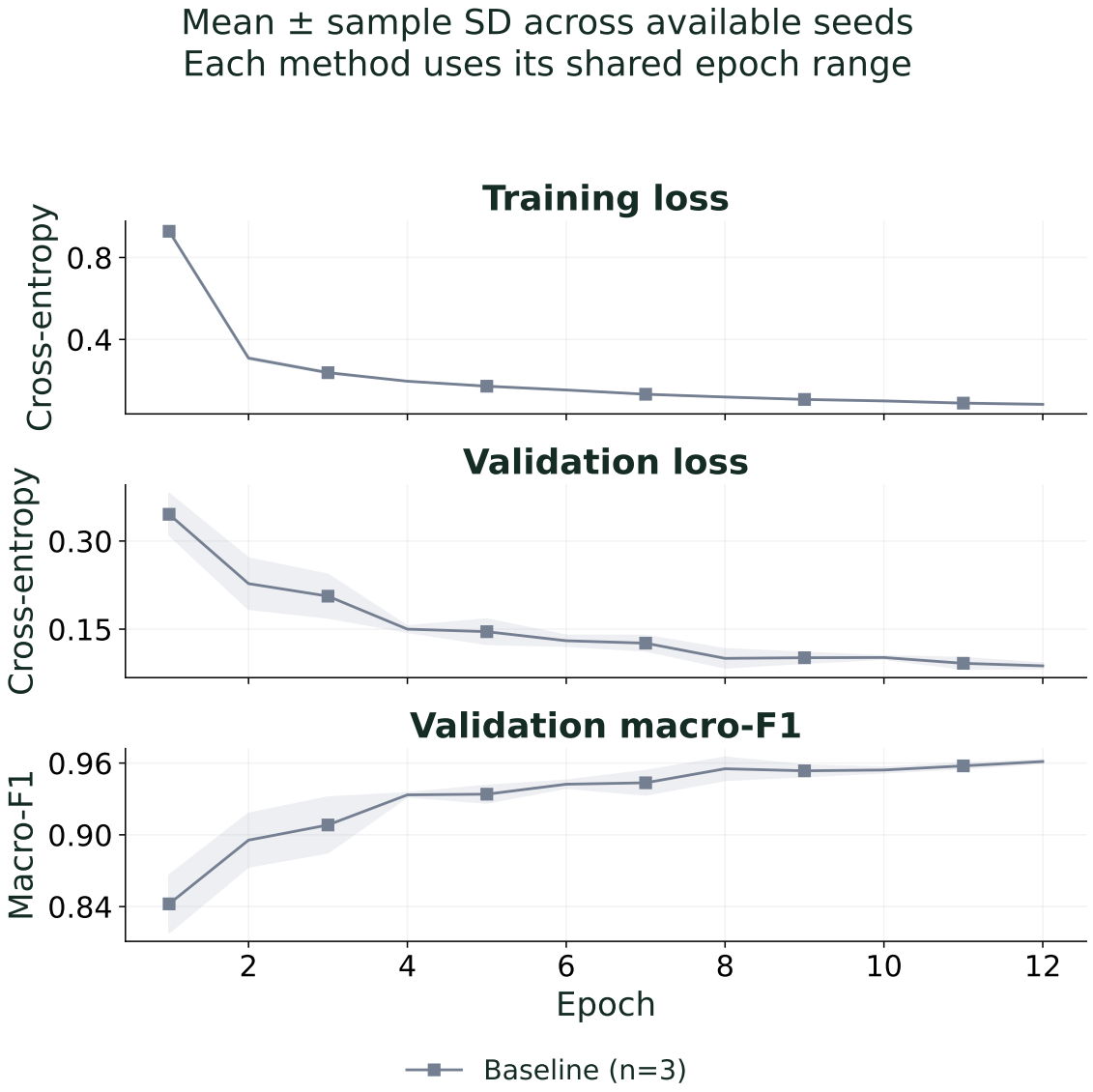


Figure 7.9: Final-training curves use only epochs shared by all seeds within each method; stopped runs are not extrapolated. Raw losses use recipe-specific label smoothing and are different objectives; compare macro-F1 or accuracy across recipes.

Table 7.4: Seed-zero bootstrap intervals. These are separate from variation across training seeds.

Model (seed 0)	Accuracy 95% CI (%)	Macro-F1 95% CI (%)
Pending: required experiment evidence is not available.		

Table 7.5: Seed-zero calibration and mean corruption performance, generated from final evaluation.

Model	Mean corruption acc. (%)	ECE (%)	Latency (ms)
Pending: required experiment evidence is not available.			

Figure awaiting its measured source artifacts: `fig_reliability`.

Figure awaiting its measured source artifacts: `fig_gradcam_grid`.

Grad-CAM maps are useful for identifying suspicious background reliance or contrasting attention patterns. They do not prove that the network has learned a biological diagnosis. The supplementary prediction gallery and embedding projection are generated as standalone repository artifacts after held-out evaluation; the confusion matrices, per-class scores, and six Grad-CAM cases provide the main report's error analysis. Each qualitative observation should be checked against the displayed image and saved prediction rather than inferred from a visually attractive plot.

CHAPTER 8

Comparative Performance Analysis

8.1 DE versus random search

Identical decoding, proxy data, fitness, budgets, and validation reranking compare differential population updates with independent sampling.

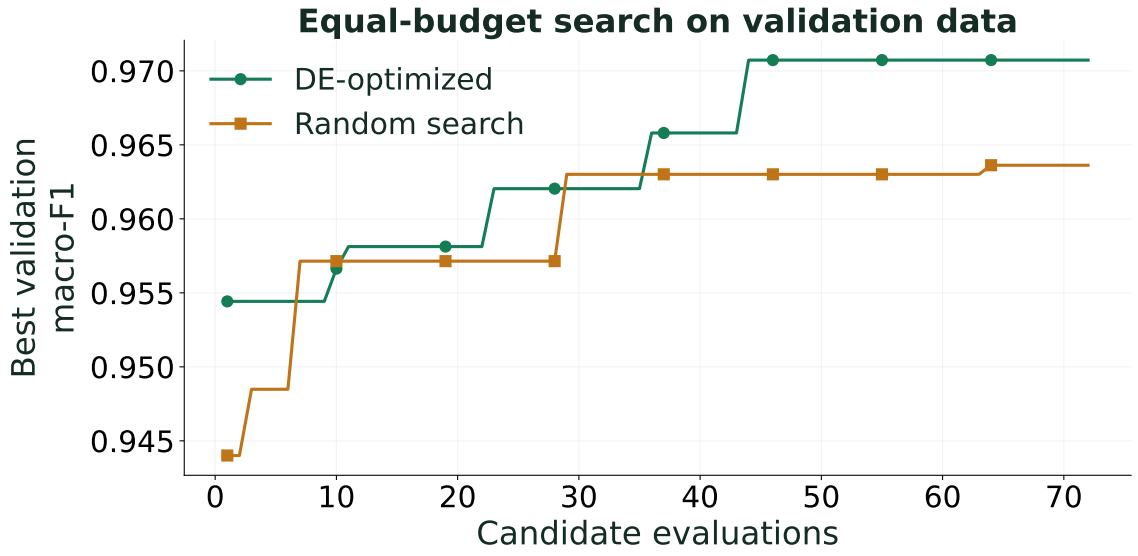


Figure 8.1: Best-so-far proxy validation macro-F1 against candidate evaluations for DE and random search.

Final retraining and evaluation remain pending; a proxy curve alone cannot establish the final model ranking.

Baseline differences concern the entire recipe, including Adam versus AdamW; they are not gene ablations. DE versus random search isolates proposal strategy more directly.

There is one CNN search per method. Final training seeds measure variation conditional on selected recipes, not across independent searches. Analytic-benchmark replication does not apply to this comparison.

Published PlantVillage studies [4, 5, 6] differ in labels and splits; evolutionary architecture studies [16, 19] differ in search spaces and resources. They provide context, not a directly comparable leaderboard.

8.2 Algorithm benchmarks and compute cost

Median winners depend on landscape: DE on Rosenbrock, PSO on Sphere/Ackley, and jDE on Rastrigin/Griewank. Random search has the largest medians. Near-zero differences indicate numerical convergence, not universal practical superiority.

Table 8.1: Final benchmark errors: median and best over all recorded runs.

Function	DE	jDE	GA	PSO	Random search
Sphere	2.59×10^{-15} 3.456×10^{-16}	1.27×10^{-23} 6.087×10^{-25}	0.001 055 0.000 321 7	2.59×10^{-26} 8.457×10^{-28}	11.66 4.67
Rosenbrock	0.4724 0.000 577 7	3.332 1.141	7.656 1.647	2.657 0.049 64	8991 2511
Rastrigin	30.01 11.68	2.053×10^{-12} 1.421×10^{-14}	3.298 0.4066	6.467 2.985	64.36 46.7
Ackley	5.464×10^{-7} 1.599×10^{-7}	2.02×10^{-11} 3.578×10^{-12}	0.447 0.2319	2.329×10^{-12} 3.877×10^{-13}	16.63 13.42
Griewank	0.4567 0.1779	1.286×10^{-5} 1.846×10^{-12}	0.6218 0.3308	0.082 41 0.022 15	41.02 17.04

Each cell: median (upper), best (lower); lower objective error is better.

Table 8.2: Exploratory two-sided rank-sum tests; negative z favors DE.

Function	DE vs jDE	DE vs GA	DE vs PSO	DE vs random search
Sphere	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$
Rosenbrock	$z = -6.121$ $p = 9.313 \times 10^{-10}$	$z = -6.55$ $p = 5.773 \times 10^{-11}$	$z = -5.485$ $p = 4.134 \times 10^{-8}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$
Rastrigin	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = 6.55$ $p = 5.773 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$
Ackley	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$
Griewank	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -5.337$ $p = 9.44 \times 10^{-8}$	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$

Exploratory rank-sum tests; no tie correction despite tied errors.

Two-sided p-values, unadjusted for multiple comparisons; negative z favors DE.

Tests are unadjusted for multiple comparisons and not tie-corrected despite tied errors. Approximate p-values are exploratory, not confirmatory. Interpret error magnitudes and distributions under these fixed budgets.

Table 8.3: Measured training, model, and inference cost.

Model	Trainable params	Size (MiB)	Training (s)	Latency (ms)
Pending: required experiment evidence is not available.				

Benchmark invocation took 105.34 seconds; summed optimizer times and CNN training are separate. Comparisons require matching devices, resolution, batches, and cache state.

CHAPTER 9

Applications and Case Studies

9.1 Offline diagnosis and agricultural workflows

The demo presents baseline and DE predictions, top-class scores, and genuine Grad-CAM. Controlled corruptions expose sensitivity, while a separate Evolution Lab explains population search. These features support classroom demonstrations and a possible agricultural-extension workflow. Greenhouse monitoring, drone imagery, and outbreak mapping are prospective applications requiring new image distributions, spatial context, and evaluation. The current model supplies neither treatment prescriptions nor validated outbreak estimates.

9.2 Inspecting the project's test images

The saved prediction and Grad-CAM grids provide an auditable case study. First inspect a correct example: compare the two labels, confidence scores, and attended regions against visible symptoms. Then inspect a disagreement or error, when available: identify the predicted alternative and potential background cues. A third image under corruption tests whether the decision remains stable. These steps use the same image across models and retain failures rather than selecting only attractive examples. Maps indicate model evidence, not causal proof or lesion annotations [23].

9.3 PlantVillage Nuru and transferable lessons

Mrisho et al. evaluate PlantVillage Nuru for cassava symptoms and describe offline smartphone operation [41]. Image acquisition, including the number of examined leaves, affects performance. Those cassava findings do not validate this tomato model; they show why field assessment must include users, devices, capture procedure, and unknown-input handling.

9.4 Optimization beyond this application

DE surveys describe scientific and engineering optimization, including design and energy problems [34, 35]. The common requirement is a candidate objective that is expensive or inconvenient to differentiate. DEHB combines evolutionary sampling with resource allocation [19]; hybrid DE has also been studied for CNN architecture design [18]. These are published applications, not additional experiments reproduced here.

CHAPTER 10

Conclusion

This project connects a from-scratch Differential Evolution implementation to an auditable transfer-learning workflow and an offline interactive demonstration. The fixed backbone is EfficientNet-B0, the dataset is PlantVillage's tomato subset, and the decision vector contains training and classifier/fine-tuning parameters. The evaluation design protects the held-out split from model selection and compares adaptive search with an equal-budget random control.

Data preparation retained 18,146 usable images after documented cleaning. The analytic study completed 750 optimizer runs with 15,000,000 objective calls. The median DE Sphere error was 2.59×10^{-15}. These measured artifacts validate important components independently of the final CNN comparison.

The final CNN comparison has not yet completed in this build. Its accuracy, statistical comparison, and final qualitative conclusions are therefore not asserted. Completion of data and analytic benchmarks must not be presented as completion of the final neural experiment.

The data objective is supported by committed manifests and cleaning records. The algorithm objective is supported by operator tests and independent benchmarks. The model-comparison objective is supported only when the frozen-protocol final metrics are present. Reproducibility is supported by configuration, seeds, checkpoints, generated tables, and explicit source provenance. The principal external-validity limitation remains the controlled nature of PlantVillage; independent field evaluation is required before broader deployment claims.

CHAPTER 11

Future Work

11.1 Evaluate the intended field distribution

A priority is independent evaluation on field images with verified tomato class mapping. PlantDoc and Field-PV research illustrate why this matters [8, 9]. Images from new devices, locations, symptom severities, and multiple simultaneous stresses should be evaluated under a fresh protocol. An unknown-input rejection mechanism should be validated rather than equated with a low softmax threshold.

11.2 Improve search efficiency without weakening controls

Multi-fidelity allocation could spend more resources on promising candidates while terminating weak recipes early. DEHB offers an established example [19]; SHADE and L-SHADE motivate adaptive parameter and population strategies [37, 38]. Such additions require matched resource accounting and new validation-only comparisons. They must not retroactively change the baseline or use test results to select an optimizer.

11.3 Optimize the deployment trade-off

An accuracy-only objective can be extended to explicitly account for latency, model size, or energy. A multi-objective search should report a Pareto trade-off rather than hide a weighting decision inside a single score. On-device export would need agreement checks against the source checkpoints and latency measurements on the actual target hardware. Mobile architecture literature offers useful context [11, 12], while the current submitted system retains its fixed EfficientNet-B0 backbone.

11.4 Improve evidence and feedback

Further work can examine calibration under shift, group-level uncertainty estimates, and expert-reviewed explanations. Farmer or extension-worker feedback could guide collection of difficult cases, but any iterative data acquisition would require new, protected evaluation data. These are proposed studies, not results of the current project.

CHAPTER 12

References

[1] S. Savary, L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts, and A. Nelson, "The global burden of pathogens and pests on major food crops," *Nature Ecology & Evolution*, vol. 3, no. 3, pp. 430–439, 2019. [Online]. Available: https://doi.org/10.1038/s41559-018-0793-y

[2] Ministry of Agriculture and Farmers Welfare, "Horticulture area and production estimates, released 17 march 2026," Press Information Bureau, Government of India, 2026, official government source; no DOI assigned. [Online]. Available: https://www.pib.gov.in/PressReleasePage.aspx?PRID=2241379&lang=1&reg=3

[3] D. P. Hughes and M. Salathe, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv preprint, 2015. [Online]. Available: https://doi.org/10.48550/arxiv.1511.08060

[4] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, 2016. [Online]. Available: https://doi.org/10.3389/fpls.2016.01419

[5] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018. [Online]. Available: https://doi.org/10.1016/j.compag.2018.01.009

[6] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019. [Online]. Available: https://doi.org/10.1016/j.compag.2018.03.032

[7] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018. [Online]. Available: https://doi.org/10.1016/j.compag.2018.02.016

[8] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A Dataset for Visual Plant Disease Detection," in *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*. ACM, 2020, pp. 249–253. [Online]. Available: https://doi.org/10.1145/3371158.3371196

[9] P. Gui, W. Dang, F. Zhu, and Q. Zhao, "Towards automatic field plant disease recognition," *Computers and Electronics in Agriculture*, vol. 191, p. 106523, 2021. [Online]. Available: https://doi.org/10.1016/j.compag.2021.106523

[10] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," arXiv preprint, 2019. [Online]. Available: https://doi.org/10.48550/arxiv.1905.11946

[11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," arXiv preprint, 2018. [Online]. Available: https://doi.org/10.48550/arxiv.1801.04381

[12] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," arXiv preprint, 2019. [Online]. Available: https://doi.org/10.48550/arxiv.1905.02244

[13] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" arXiv preprint, 2014. [Online]. Available: https://doi.org/10.48550/arxiv.1411.1792

[14] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," arXiv preprint, 2012. [Online]. Available: https://doi.org/10.48550/arxiv.1206.2944

[15] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," arXiv preprint, 2016. [Online]. Available: https://doi.org/10.48550/arxiv.1603.06560

[16] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized Evolution for Image Classifier Architecture Search," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 4780–4789, 2019. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.33014780

[17] B. Wang, Y. Sun, B. Xue, and M. Zhang, "Evolving Deep Convolutional Neural Networks by Variable-Length Particle Swarm Optimization for Image Classification," in *2018 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2018, pp. 1–8. [Online]. Available: https://doi.org/10.1109/cec.2018.8477735

[18] ——, "A Hybrid Differential Evolution Approach to Designing Deep Convolutional Neural Networks for Image Classification," in *AI 2018: Advances in Artificial Intelligence*. Springer International Publishing, 2018, pp. 237–250. [Online]. Available: https://doi.org/10.1007/978-3-030-03991-2_24

[19] N. Awad, N. Mallik, and F. Hutter, "DEHB: Evolutionary Hyberband for Scalable, Robust and Efficient Hyperparameter Optimization," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 2021, pp. 2147–2153. [Online]. Available: https://doi.org/10.24963/ijcai.2021/296

[20] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

[21] T. G. Dietterich, "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998. [Online]. Available: https://doi.org/10.1162/089976698300017197

[22] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," arXiv preprint, 2017. [Online]. Available: https://doi.org/10.48550/arxiv.1706.04599

[23] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in *2017 IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2017, pp. 618–626. [Online]. Available: https://doi.org/10.1109/iccv.2017.74

[24] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," arXiv preprint, 2019. [Online]. Available: https://doi.org/10.48550/arxiv.1903.12261

[25] J. H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. The MIT Press, 1992. [Online]. Available: https://doi.org/10.7551/mitpress/1090.001.0001

[26] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4. IEEE, 1995, pp. 1942–1948. [Online]. Available: https://doi.org/10.1109/icnn.1995.488968

[27] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996. [Online]. Available: https://doi.org/10.1109/3477.484436

[28] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983. [Online]. Available: https://doi.org/10.1126/science.220.4598.671

[29] X.-S. Yang, "Firefly Algorithms for Multimodal Optimization," in *Stochastic Algorithms: Foundations and Applications*. Springer Berlin Heidelberg, 2009, pp. 169–178. [Online]. Available: https://doi.org/10.1007/978-3-642-04944-6_14

[30] D. Simon, "Biogeography-Based Optimization," *IEEE Transactions on Evolutionary Computation*, vol. 12, no. 6, pp. 702–713, 2008. [Online]. Available: https://doi.org/10.1109/tevc.2008.919004

[31] D. Karaboga and B. Basturk, "A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm," *Journal of Global Optimization*, vol. 39, no. 3, pp. 459–471, 2007. [Online]. Available: https://doi.org/10.1007/s10898-007-9149-x

[32] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey Wolf Optimizer," *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014. [Online]. Available: https://doi.org/10.1016/j.advengsoft.2013.12.007

[33] R. Storn and K. Price, "Differential Evolution – A Simple and Efficient Heuristic for global Optimization over Continuous Spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997. [Online]. Available: https://doi.org/10.1023/a:1008202821328

[34] S. Das and P. N. Suganthan, "Differential Evolution: A Survey of the State-of-the-Art," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2011. [Online]. Available: https://doi.org/10.1109/tevc.2010.2059031

[35] S. Das, S. S. Mullick, and P. Suganthan, "Recent advances in differential evolution – An updated survey," *Swarm and Evolutionary Computation*, vol. 27, pp. 1–30, 2016. [Online]. Available: https://doi.org/10.1016/j.swevo.2016.01.004

[36] J. Brest, S. Greiner, B. Boskovic, M. Mernik, and V. Zumer, "Self-Adapting Control Parameters in Differential Evolution: A Comparative Study on Numerical Benchmark Problems," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 6, pp. 646–657, 2006. [Online]. Available: https://doi.org/10.1109/tevc.2006.872133

[37] R. Tanabe and A. Fukunaga, "Success-history based parameter adaptation for Differential Evolution," in *2013 IEEE Congress on Evolutionary Computation*. IEEE, 2013, pp. 71–78. [Online]. Available: https://doi.org/10.1109/cec.2013.6557555

[38] R. Tanabe and A. S. Fukunaga, "Improving the search performance of SHADE using linear population size reduction," in *2014 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2014, pp. 1658–1665. [Online]. Available: https://doi.org/10.1109/cec.2014.6900380

[39] K. Sörensen, "Metaheuristics—the metaphor exposed," *International Transactions in Operational Research*, vol. 22, no. 1, pp. 3–18, 2015. [Online]. Available: https://doi.org/10.1111/itor.12001

[40] M. Jamil and X. S. Yang, "A literature survey of benchmark functions for global optimisation problems," *International Journal of Mathematical Modelling and Numerical Optimisation*, vol. 4, no. 2, p. 150, 2013. [Online]. Available: https://doi.org/10.1504/ijmmno.2013.055204

[41] L. M. Mrisho, N. A. Mbilinyi, M. Ndalahwa, A. M. Ramcharan, A. K. Kehs, P. C. McCloskey, H. Murithi, D. P. Hughes, and J. P. Legg, "Accuracy of a Smartphone-Based Object Detection Model, PlantVillage Nuru, in Identifying the Foliar Symptoms of the Viral Diseases of Cassava–CMD and CBSD," *Frontiers in Plant Science*, vol. 11, 2020. [Online]. Available: https://doi.org/10.3389/fpls.2020.590889

CHAPTER 13

GitHub Repo Link

https://github.com/Suraj-B12/BIO-INSPIRED-ALGO---AAT

The source revision recorded when the report tables were generated is `1cca795ad5d9` (provenance: Project Git HEAD). Exact delivered-file hashes are recorded in the delivery manifest. The repository contains the configuration, numbered execution scripts, tests, result artifacts, figure generators, report sources, presentation generator, and offline demo. Large raw data and temporary training files are excluded from the source distribution.

Table 13.1: Repository map and reproducibility entry points.

Location	Purpose
`bioalgo/`	Data/model helpers, training, DE and benchmark algorithms
`scripts/`	Numbered data, experiment, evaluation, and artifact stages
`tests/`	Operator, data, model, service, and smoke verification
`results/`	Saved experiment evidence and source-provenance records
`figures/`	Reproducible plots generated from committed evidence
`report/`, `slides/`	Academic deliverables and editable sources
`demo/`	Local offline diagnosis and Evolution Lab

After creating a Python environment and installing the documented dependencies, the cross-platform entry point is `python run_all.py`. A synthetic smoke run uses `python run_all.py -smoke`. Existing result figures can be reproduced without retraining through `python scripts/09_figures.py`; tables are regenerated by `python scripts/10_make_tables.py`. Start the installed demo with `python demo/serve.py` and open its printed local address.

The workflow configuration targets macOS, Linux, and Windows. A CI configuration is not itself proof that every operating-system run passed; consult the actual repository Actions run and the verification report for observed status. Full experiment reproduction also requires the original data source and pretrained weights to be available during setup. Once installed and equipped with the final checkpoints, the demo operates offline.